



CYBERSECURITY INTERNSHIP REPORT

Comprehensive Report – Task 2

Internship conducted at:

Redynox Cybersecurity

<https://redynox.theredusers.tech/>

Role: Cyber Security Intern

Duration: 21 September 2025 – 20 October 2025

Location: Remote

Submitted by:

Mohammed Ryad DERMOUCHE

Date: **October 20, 2025**



Document Information

Document identification

Document Reference:	REDYNOX-INTERNSHIP-REPORT-V1.00
Version:	1.0
Date of Issue:	October 20, 2025
Author:	Mohammed Ryad DERMOUCHE
Organization:	Redynox Cybersecurity
Program Duration:	21 September – 20 October 2025
Confidentiality:	Internal – For academic and evaluation use only

Verification elements

Validated by:	Ankit MATHUR (Redynox Lead Supervisor)
Validation Date:	20/10/2025
File Type:	PDF – Digital submission
Reviewers:	Internship Supervisor (Redynox)



Contents

1	Introduction	4
1.1	Context	4
1.2	Objectives	4
1.3	Structure of the Report	4
2	Task 2 – Introduction to Web Application Security	5
2.1	Objective	5
2.2	Skills	5
2.3	Tools	5
2.4	Guidelines	5
2.4.1	What is a Web Application?	6
2.4.2	OWASP	6
2.4.3	WebGoat	6
2.5	bWAPP / OWASP ZAP	7
2.5.1	Quick start	7
2.5.2	Browser / Proxy configuration	8
2.5.3	Automatic scan	9
2.5.4	Manual testing steps	11



1 Introduction

1.1 Context

This report summarizes my participation in the **Redynox Cybersecurity Internship Program**, held remotely from **21 September 2025 to 20 October 2025**. The internship provided hands-on exposure to core areas of cybersecurity including network security, vulnerability assessment, penetration testing, and incident response. The work was conducted under the supervision of **Ankit Mathur**, who guided interns through practical projects aligned with industry best practices.

1.2 Objectives

- Strengthen fundamental and applied cybersecurity knowledge.
- Work on real-world security problems and simulation projects.
- Document results and technical configurations professionally.
- Develop teamwork, autonomy, and analytical capabilities.

1.3 Structure of the Report

The report compiles all tasks accomplished during the program, organized into thematic chapters. Each task includes its goals, implementation, and learning outcomes.



2 Task 2 – Introduction to Web Application Security

2.1 Objective

Learn about common web application vulnerabilities by analysing a simple vulnerable web application. This task focuses on understanding how attackers exploit weaknesses such as input validation flaws and how to mitigate them through secure coding and configuration practices.

2.2 Skills

- Basic Web Security
- Vulnerability Identification
- Secure Coding and Mitigation Practices

2.3 Tools

- OWASP ZAP
- WebGoat (or another intentionally vulnerable web application)
- Web Browser (Firefox / Chrome)
- Local Virtual Environment (Kali Linux or Windows)

2.4 Guidelines



2.4.1 What is a Web Application?

A *web application* is a software system that runs on a web server and is accessed through a web browser or HTTP API. Typical components include a frontend (HTML/CSS/-JavaScript), a backend (server-side logic), and a database. Web applications expose endpoints (pages, forms, APIs) which, if not properly secured, can be abused by attackers to perform data theft, session hijacking, remote code execution, or other malicious actions.

2.4.2 OWASP

OWASP stands for the *Open Web Application Security Project*. It is a worldwide, non-profit organization focused on improving the security of software. OWASP produces community-driven resources such as the OWASP Top Ten (a list of the most critical web application security risks), tools, documentation, and learning labs.

OWASP Zed Attack Proxy (ZAP)

OWASP ZAP (Zed Attack Proxy) is an open-source web application security scanner and intercepting proxy maintained by OWASP. ZAP is used to:

- passively and actively scan web applications for common vulnerabilities (e.g. SQL Injection, XSS, CSRF),
- intercept and inspect HTTP(S) requests and responses,
- automate vulnerability discovery via spidering and active scanning,
- assist manual testing by allowing request modification and replay.

2.4.3 WebGoat

WebGoat is an intentionally vulnerable web application created by OWASP for learning and practice. It provides a safe, local environment with exercises that demonstrate common vulnerabilities and exploitation techniques (for example: SQL Injection, Cross-Site Scripting, and CSRF). WebGoat is ideal for training because it explains vulnerabilities, allows controlled exploitation, and suggests mitigation strategies. `localhost:8080/WebGoat`



2.5 bWAPP / OWASP ZAP

2.5.1 Quick start

```
# Install Docker
sudo apt install -y docker.io

# Start bWAPP (target)
sudo docker run -d -p 80:80 hackersploit/bwao-docker

# Access bWAPP
# Open browser and visit http://127.0.0.1 and login to bWAPP

# Stop Docker (if needed)
sudo docker ps -a          # check container name/id
docker container stop <name_or_id>

# Install ZAP (zapproxy)
sudo apt install -y zapproxy
# Or run ZAP Docker:
# docker run -u zap -p 8081:8080 owasp/zap2docker-stable
```

Screenshot: Quick start / Installation

I include a screenshot showing the terminal output for Docker/ZAP installation and the bWAPP container list.

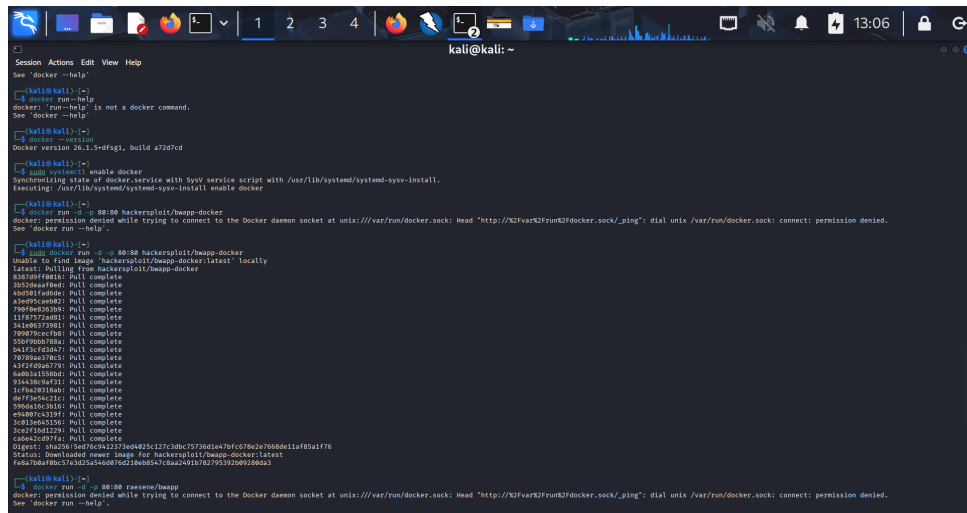


Figure 2.1: Installation and container start .

2.5.2 Browser / Proxy configuration

- I configured Firefox to route traffic through ZAP so ZAP can analyse and scan requests/responses.
- I edited `about:config` and set the relevant local interception hijacking flag to `true`.

Screenshot: Browser / Proxy configuration

I include screenshots of my Firefox proxy settings and the modified `about:config` entry.

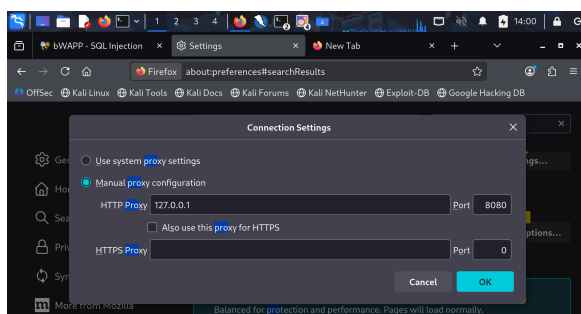


Figure 2.2: Firefox proxy settings.

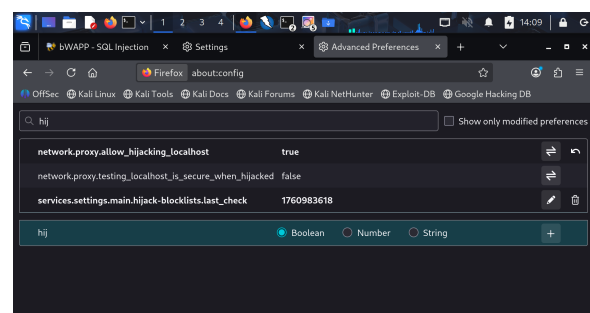


Figure 2.3: `about:config` flag I changed .



2.5.3 Automatic scan

1. I started ZAP .

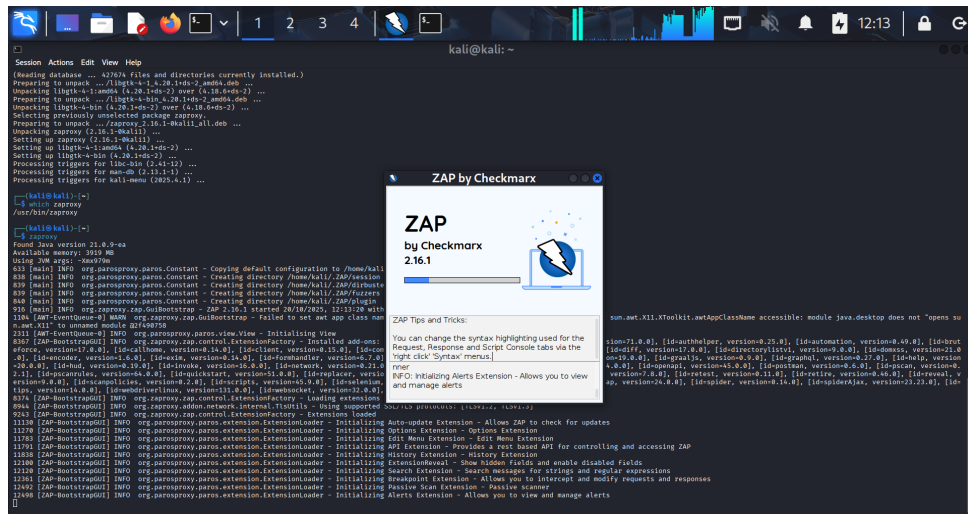


Figure 2.4: Zaproxy.

2. I launched an Active Scan in ZAP on the discovered context (default rules).

Screenshot: ZAP Active Scan

I include a screenshot of the Active Scan progress panel showing scanned URLs and status.

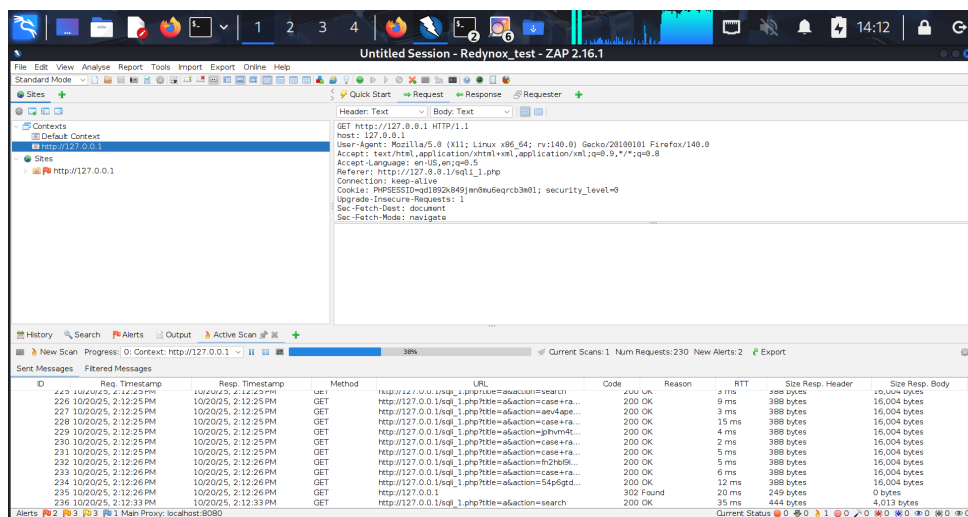


Figure 2.5: ZAP Active Scan progress .

3. I captured the ZAP alerts generated after running the Active Scan. These included SQL advanced injection (High), XSS advanced (High), and CSRF (Medium) alerts as required by the task.

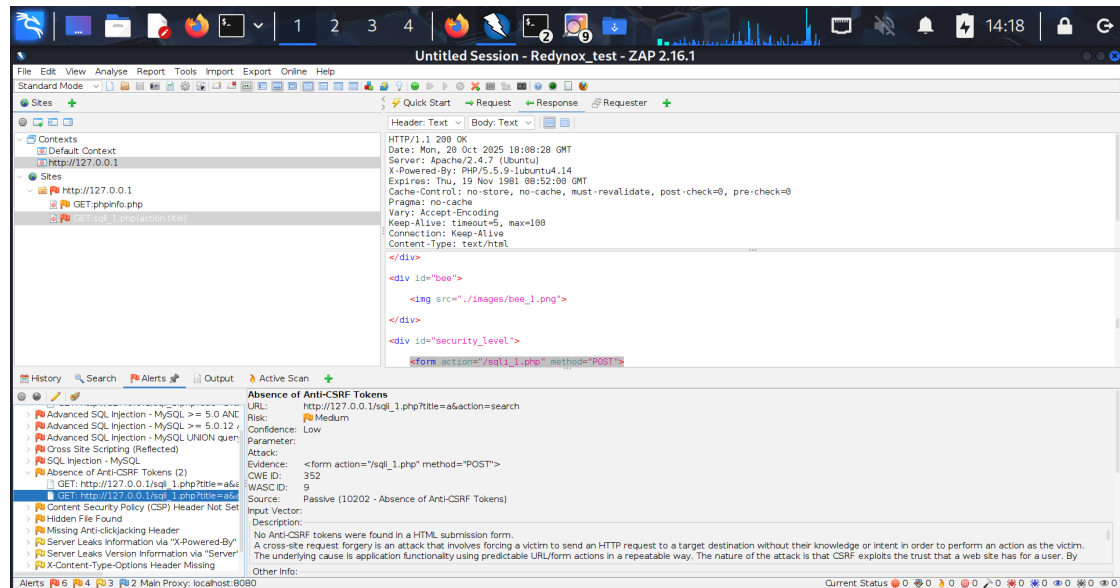


Figure 2.9: Cross-Site Request Forgery (CSRF) — missing anti-CSRF token (Medium severity).

2.5.4 Manual testing steps

Manual SQL injection test

(a) I opened the bWAPP SQL injection exercise page.

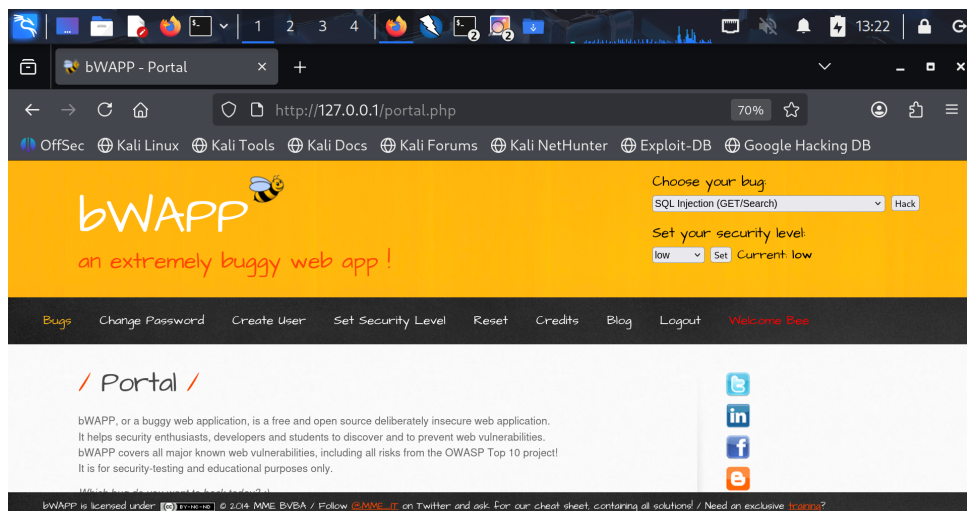


Figure 2.10: bWAPP SQL injection exercise .

(b) I edited and replayed the request inside ZAP to capture the exact HTTP request/response.

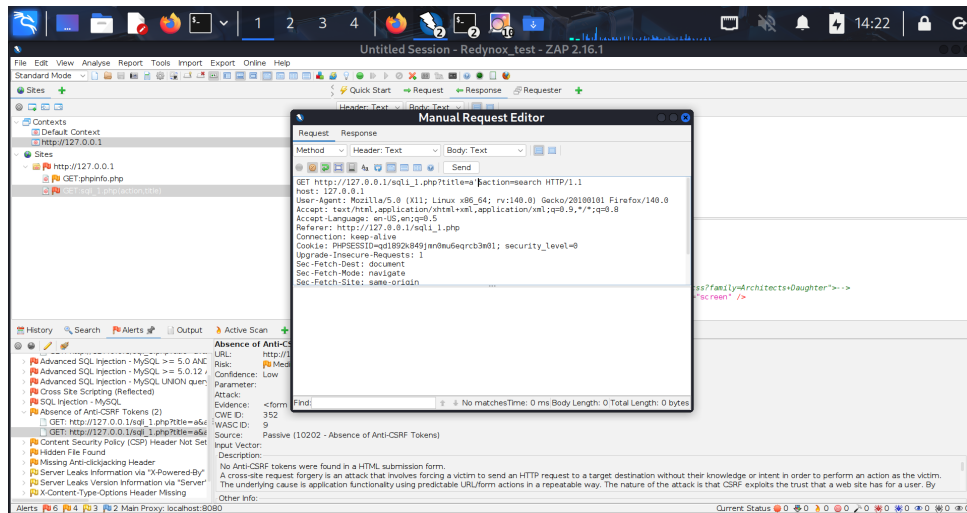


Figure 2.11: ZAP request editor showing SQLi request/response (my screenshot).

Quick Mitigations — SQL Injection, XSS, CSRF

SQL Injection (SQLi)

What it is: attacker injects SQL through input fields to read or modify the database.

Simple gestures:

- **Use prepared statements / parameterized queries** for every DB access (no string concatenation).
- **Use an ORM** or database library that enforces parameterization.
- **Validate input types and length** (e.g., numeric IDs must be numbers; max length).
- **Least-privilege DB accounts:** app user should only have needed rights (SELECT/INSERT/UPDATE limited to required tables).
- **Avoid detailed DB errors in production:** log full errors internally, show generic messages to users.
- **Whitelist input where possible:** when values are from a finite set (country codes, page IDs), validate against the whitelist.



Cross-Site Scripting (XSS)

What it is: attacker injects malicious script that runs in other users' browsers.

Simple gestures (do these now):

- **Output-encode/escape** all untrusted data when rendering HTML (use context-aware escaping: HTML, attribute, JS, URL).
- **Sanitize input** for rich text only if you must allow HTML (use a robust sanitizer library).
- **Enable framework auto-escaping:** default templating features (e.g., Django, Jinja, React) already escape—use them.
- **Set Content Security Policy (CSP)** with strict defaults (disallow inline scripts, restrict script sources).
- **Mark cookies HttpOnly and Secure**, to make stealing via XSS harder.
- **Limit user-supplied HTML:** prefer markdown/plain text and convert server-side with safe renderer.

Cross-Site Request Forgery (CSRF)

What it is: attacker forces an authenticated user's browser to send unwanted requests (state changes) to the app.

Simple gestures (do these now):

- **Use anti-CSRF tokens:** include a secure, per-session/per-form token and verify it server-side on state-changing requests.
- **Require unsafe methods for changes:** use POST/PUT/DELETE for actions, avoid GET for state changes.
- **Set cookie SameSite=strict or lax** where compatible to reduce cross-site cookie sending.
- **Verify Origin/Referer headers** for critical actions (reject requests with missing or wrong origin when possible).
- **Require re-auth or 2FA** for very sensitive operations (password change, funds transfer).



Final Illustration



OWASP ZAP



Redynox Cyber Security Internship

“Every vulnerable application is a chance to learn — every scan a step closer to secure design.”



— End of Task 2 Report —